LAPORAN PRAKTIKUM #2
FSM String Validator

Departemen Teknik Informatika
Institut Teknologi Sepuluh Nopember

Mata Kuliah : Otomata
Semester: Genap 2025/2026
Kelompok: B03

Berwyn Rafif Alvaro	5025241029
Mahendra Agung Darmawan	5025241032
Agil Lukman Hakim Muchdi	5025241037

Surabaya, 2026

Daftar Isi

1. Pendahuluan 3
1.1 Latar Belakang 3
1.2 Tujuan Praktikum 3
1.3 Ruang Lingkup 3
2. Dasar Teori 4
2.1 Finite State Machine (FSM) 4
2.2 Bahasa Formal L 4
2.3 Diagram Transisi 4
3. Perancangan Program 5
3.1 Arsitektur Sistem 5
3.2 Definisi State dan Transisi 5
3.3 Algoritma Simulasi FSM 5
3.3.1 Tabel Transisi 5
3.3.2 Alur Eksekusi FSM 5
3.4 Antarmuka Pengguna 6
4. Implementasi 6
4.1 Lingkungan Pengembangan 6
4.2 Struktur Kode 6
4.3 Fitur yang Diimplementasikan 6
5. Hasil dan Pengujian 7
5.1 Kasus Uji: String Diterima 7
5.2 Kasus Uji: String Ditolak 8
5.3 Ringkasan Pengujian 9
6. Pembahasan 9
6.1 Keunggulan Implementasi 9
6.2 Kompleksitas Algoritma 9
7. Kesimpulan 10
LAMPIRAN A. Tautan Google Collab 10
LAMPIRAN B. Source Code Lengkap 11

1. Pendahuluan

1.1 Latar Belakang

Finite State Machine (FSM) adalah model komputasi yang banyak digunakan dalam teori bahasa formal dan automata. FSM merupakan fondasi dari berbagai aplikasi nyata, mulai dari validasi input pada sistem kompiler hingga pengendalian alur logika dalam sistem tertanam. Pemahaman terhadap FSM sangat penting dalam studi Teori Bahasa dan Automata karena konsep ini langsung berkaitan dengan kemampuan suatu mesin dalam mengenali bahasa formal tertentu.

Pada Tugas Praktikum #2 ini, kelompok kami diminta untuk mengimplementasikan sebuah program yang mensimulasikan FSM dalam menentukan keanggotaan sebuah string terhadap bahasa $L = \{ x \in (0 + 1)^+ \mid$ karakter terakhir pada string x adalah 1 dan x tidak memiliki substring 00}. Program ini diharapkan mampu melacak perpindahan state dan memberikan keluaran yang jelas mengenai apakah sebuah string diterima atau ditolak oleh mesin.

1.2 Tujuan Praktikum

1. Membangun program komputer yang dapat mensimulasikan FSM berdasarkan diagram transisi yang diberikan.
2. Menentukan apakah sebuah string biner merupakan anggota dari bahasa $L = \{ x \in (0 + 1)^+ \mid$ karakter terakhir pada string x adalah 1 dan x tidak memiliki substring 00}
3. Menampilkan jejak perpindahan state (trace) untuk setiap karakter yang diproses.
4. Merancang user interface yang mudah digunakan sehingga pengguna dapat dengan mudah memasukkan string yang akan diidentifikasi keanggotaannya.

1.3 Ruang Lingkup

Program yang dikembangkan beroperasi pada alfabet biner $\Sigma = \{0, 1\}$ dan mengimplementasikan FSM dengan empat state (S, A, B, C) sesuai diagram yang diberikan pada soal. Input yang valid hanya berupa karakter '0' dan '1'. Program juga dilengkapi dengan penanganan input tidak valid dan fitur pengulangan untuk kemudahan pengujian.

2. Dasar Teori

2.1 Finite State Machine (FSM)

Finite State Machine (FSM) atau Mesin Keadaan Hingga adalah model matematis dari sebuah sistem komputasi yang memiliki jumlah state yang terbatas. Secara formal, sebuah FSM deterministik (DFA) didefinisikan sebagai tuple M = (Q, Σ, δ, q0, F), di mana: (1) Q adalah himpunan state yang terbatas, (2) Σ adalah alfabet input, (3) δ : Q × Σ → Q adalah fungsi transisi, (4) q0 ∈ Q adalah state awal, dan (5) F ⊆ Q adalah himpunan state penerima. Sebuah string diterima oleh FSM jika dan hanya jika setelah memproses seluruh karakter input, mesin berada pada salah satu state dalam himpunan F. Jika tidak, string tersebut ditolak.

2.2 Bahasa Formal L

Bahasa yang dikenali oleh FSM pada tugas ini adalah:

$$\mathbf{L} = \{\mathbf{x} \in (\mathbf{0} + \mathbf{1})^{+} \mid \textbf{karakter terakhir pada string x adalah 1 dan x tidak memiliki substring 00}\}$$

Dengan kata lain, suatu string diterima jika memenuhi dua syarat sekaligus: (1) string berakhir dengan karakter '1', dan (2) string tidak mengandung dua karakter '0' yang berurutan (substring '00'). String juga harus memiliki panjang minimal satu karakter karena domain yang digunakan adalah $(0 + 1)^{+}$.

2.3 Diagram Transisi

FSM pada tugas ini memiliki empat state dengan rincian sebagai berikut:

State	Deskripsi	State Penerima
S	State awal (start state)	Tidak
A	State setelah membaca '0' dari S atau B	Tidak
B	State setelah membaca '1' (accepting state)	Ya
C	State trap, masuk saat membaca '0' dari A (mengindikasikan adanya '00')	Tidak

Tabel 1: Deskripsi State pada FSM

Fungsi transisi δ didefinisikan sebagai berikut: dari state S, membaca '0' menghasilkan state A dan membaca '1' menghasilkan state B. Dari state A, membaca '0' menghasilkan state C (trap) dan membaca '1' menghasilkan state B. Dari state B, membaca '0' menghasilkan state A dan membaca '1' menghasilkan state B itu sendiri. Dari state C (trap), membaca '0' maupun '1' tetap menghasilkan state C.

3. Perancangan Program

3.1 Arsitektur Sistem

Program FSM String Validator dirancang dengan arsitektur sederhana namun terstruktur yang terdiri dari tiga komponen utama, yaitu: (1) Fungsi FSM Simulator, yaitu fungsi inti cek_fsm() yang mengeksekusi simulasi mesin dan menghasilkan trace perpindahan state; (2) Validator Input, yaitu pengecekan karakter untuk memastikan input hanya terdiri dari '0' dan '1'; serta (3) Loop Interaktif, yaitu antarmuka berbasis teks (CLI) yang memungkinkan pengguna memasukkan string berulang kali.

3.2 Definisi State dan Transisi

Program mendefinisikan empat state sebagai konstanta string: 'S' (start), 'A', 'B' (accepting), dan 'C' (trap). Fungsi transisi diimplementasikan melalui struktur kondisional if-elif yang memetakan kombinasi (state, karakter) ke state berikutnya secara eksplisit.

3.3 Algoritma Simulasi FSM

3.3.1 Tabel Transisi

Berikut adalah tabel transisi lengkap yang diimplementasikan dalam program:

State Saat Ini	Input '0'	Input '1'
S (Start)	A	B
A	C (Trap)	B
B (Accept)	A	B
C (Trap)	C	C

Tabel 2: Tabel Transisi FSM

3.3.2 Alur Eksekusi FSM

Alur eksekusi dari fungsi cek_fsm() adalah sebagai berikut:

1. Program memulai di state S dan menginisialisasi daftar history dengan [S].
2. Untuk setiap karakter c dalam string input, program memvalidasi bahwa $c \in \{0, 1\}$.
3. Fungsi transisi dieksekusi berdasarkan state saat ini dan karakter c.
4. State baru ditambahkan ke dalam daftar history.
5. Setelah semua karakter diproses, program memeriksa state akhir.
6. Jika state akhir adalah B, string DITERIMA. Selain itu, string DITOLAK beserta alasannya.

3.4 Antarmuka Pengguna

Antarmuka pengguna dibangun menggunakan input() standar Python yang berjalan pada lingkungan Google Colab. Fitur yang disediakan meliputi: prompt masukan string, tampilan trace perpindahan state dalam format 'S → A → B → ..., pesan hasil berupa *DITERIMA* atau *DITOLAK* beserta alasan penolakan, serta opsi pengulangan untuk memasukkan string berikutnya tanpa harus menjalankan ulang program.

4. Implementasi

4.1 Lingkungan Pengembangan

Komponen	Keterangan
Bahasa Pemrograman	Python 3.x
Platform	Google Colaboratory (Google Colab)
Library	Tidak ada library eksternal (pure Python)
Input/Output	Antarmuka berbasis teks (CLI via input())

Tabel 3: Lingkungan Pengembangan

4.2 Struktur Kode

Program terdiri dari dua bagian utama yang disusun secara berurutan:

1. Fungsi cek_fsm(s) – mengandung logika inti FSM: inisialisasi state, loop pemrosesan karakter, fungsi transisi, pembentukan trace, dan penentuan hasil akhir.
2. Loop Utama (while True) – mengelola interaksi pengguna, memanggil fungsi cek_fsm(), menampilkan hasil, dan menangani pilihan pengulangan.

4.3 Fitur yang Diimplementasikan

Selain fungsi FSM dasar, program dilengkapi dengan fitur-fitur berikut:

- Validasi input karakter: Program mengembalikan pesan error jika input mengandung karakter selain '0' dan '1'.
- Trace perpindahan state: Setiap langkah transisi dicatat dan ditampilkan dalam format yang mudah dibaca (misal: S -> A -> B -> B).
- Keterangan alasan penolakan: Jika string ditolak karena mengandung substring '00', program secara eksplisit menyebutkan alasan tersebut. Jika ditolak karena tidak berakhir dengan '1', alasan tersebut juga ditampilkan.
- Loop interaktif dengan opsi y/n: Pengguna dapat memilih untuk menguji string baru atau menghentikan program.

5. Hasil dan Pengujian

5.1 Kasus Uji: String Diterima

Berikut adalah beberapa contoh string yang diharapkan DITERIMA oleh FSM beserta trace-nya:

- Kasus Uji 1: String "1"

```
Masukkan string (0 dan 1): 1
TRACE FSM:
S -> B
DITERIMA
```

State awal S membaca '1' dan berpindah ke B. String berakhir di state B (accepting state), sehingga diterima. String ini memenuhi kedua syarat: berakhir dengan '1' dan tidak mengandung '00'.

- Kasus Uji 2: String "01"

```
Masukkan string (0 dan 1): 01
TRACE FSM:
S -> A -> B
DITERIMA
```

S membaca '0' → A, lalu membaca '1' → B. String diterima karena berakhir di B dan tidak memiliki substring '00'.

- Kasus Uji 3: String "101"

```
Masukkan string (0 dan 1): 101
TRACE FSM:
S -> B -> A -> B
DITERIMA
```

S → B (baca '1'), B → A (baca '0'), A → B (baca '1'). Berakhir di B, diterima.

5.2 Kasus Uji: String Ditolak

Berikut adalah contoh string yang diharapkan DITOLAK beserta alasannya:

- Kasus Uji 5: String "0" (tidak berakhir dengan '1')

```
Masukkan string (0 dan 1): 0
TRACE FSM:
S -> A
DITOLAK
Alasan: tidak berakhir dengan '1'
```

S → A (baca '0'), String hanya terdiri dari satu karakter '0' sehingga mesin berhenti di state A. String ditolak karena karakter terakhirnya adalah '0' bukan '1' sehingga syarat bahasa L tidak terpenuhi.

- Kasus Uji 6: String "001" (mengandung substring '00')

```
Masukkan string (0 dan 1): 001
TRACE FSM:
S -> A -> C -> C
DITOLAK
Alasan: mengandung substring '00'
```

S → A (baca '0'), A → C (baca '0', terdeteksi '00'), C → C (baca '1', state trap). Meskipun karakter terakhir adalah '1', string tetap ditolak karena mengandung substring '00'.

- Kasus Uji 7: String "10" (tidak berakhir dengan '1')

```
Masukkan string (0 dan 1): 10
TRACE FSM:
S -> B -> A
DITOLAK
Alasan: tidak berakhir dengan '1'
```

S → B (baca '1'), B → A (baca '0'). String berakhir di State A. Meskipun tidak terdapat substring '00', syarat bahwa karakter terakhir harus '1' tidak terpenuhi. State A hanya dicapai saat karakter terakhir yang dibaca adalah '0', sehingga string ditolak.

- Kasus Uji 8: Input tidak valid (karakter selain 0 dan 1)

```
Masukkan string (0 dan 1):10a
Input tidak valid
```

String "10a" mengandung karakter 'a' yang bukan bagian dari $\Sigma = \{0, 1\}$. *Program mengembalikan pesan error dan menghentikan proses eksekusi.*

5.3 Ringkasan Pengujian

No	String Input	Trace	Hasil	Alasan
1	1	S → B	DITERIMA	-
2	01	S → A → B	DITERIMA	-
3	101	S → B → A → B	DITERIMA	-
4	0	S → A	DITOLAK	Tidak berakhir '1'
5	001	S → A → C → C	DITOLAK	Mengandung '00'
6	10	S → B → A	DITOLAK	Tidak berakhir '1'
7	10a	-	Error	Karakter tidak valid

Tabel 4: Ringkasan Hasil Pengujian

6. Pembahasan

6.1 Keunggulan Implementasi

Beberapa keunggulan dari implementasi yang dikembangkan antara lain:

1. Kesederhanaan dan Keterbacaan, Implementasi menggunakan struktur if-elif yang linear dan mudah dipahami. Setiap blok kondisi mewakili satu state, sehingga korespondensi antara kode dan diagram transisi sangat jelas.
2. Trace yang Informatif, Program mencatat setiap perpindahan state dalam daftar history dan menampilkannya dalam format yang mudah dibaca. Hal ini memudahkan pengguna dalam memahami jalannya mesin.
3. Keterangan Alasan Penolakan, Program tidak hanya memberi tahu bahwa string ditolak, tetapi juga menjelaskan alasan spesifiknya, yaitu apakah karena mengandung substring '00' atau karena tidak berakhir dengan '1'. Ini meningkatkan transparansi dan kegunaan program.
4. Deteksi Substring '00' yang Tepat, Penggunaan state C sebagai trap state secara otomatis menangkap semua string yang mengandung substring '00', tanpa perlu pencarian string tambahan.
5. Loop Interaktif, Fitur pengulangan dengan pilihan y/n memungkinkan pengguna menguji banyak string tanpa harus menjalankan ulang sel di Colab, meningkatkan efisiensi pengujian.

6.2 Kompleksitas Algoritma

Algoritma simulasi FSM menggunakan pendekatan single-pass dengan kompleksitas waktu O(n), di mana n adalah panjang string input. Setiap karakter diproses tepat sekali, dan setiap operasi transisi dilakukan dalam waktu konstan O(1) menggunakan struktur if-elif. Kompleksitas ruang juga O(n) karena program menyimpan riwayat state untuk setiap karakter dalam daftar history.

7. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Program FSM String Validator berhasil diimplementasikan menggunakan Python murni tanpa library eksternal, dengan fungsi utama cek_fsm() yang mensimulasikan mesin secara deterministik.
2. Program berhasil mengenali bahasa $L = \{x \in (0 + 1)^+ |$ karakter terakhir adalah 1 dan x tidak memiliki substring '00' $\}$ dengan tingkat akurasi 100% pada seluruh kasus uji yang dilakukan.
3. Penggunaan state trap (C) terbukti efektif untuk menangkap semua string yang mengandung substring '00' secara otomatis tanpa memerlukan logika tambahan di luar fungsi transisi.
4. Trace perpindahan state yang ditampilkan membantu dalam memverifikasi kebenaran algoritma dan memberikan transparansi penuh terhadap proses pengambilan keputusan mesin.
5. Implementasi FSM menggunakan pendekatan single-pass menghasilkan kompleksitas waktu O(n) yang sangat efisien untuk semua panjang input.

LAMPIRAN A. Tautan Google Collab

Program FSM String Validator ini dijalankan menggunakan Google Colaboratory (Google Colab) agar dapat diakses dan dieksekusi secara langsung melalui browser tanpa instalasi lokal. Tautan notebook dapat diakses pada alamat berikut:

Platform	Google Colaboratory
Cara Akses	Buka tautan → Runtime → Run All
Tautan	https://colab.research.google.com/drive/1ekEuuJI4ZU4-ykZBc-IKXJGD3FsHpZgv?usp=sharing

Untuk menjalankan program, cukup buka notebook di Google Colab, kemudian pilih Runtime → Run All atau jalankan setiap sel secara berurutan dari atas ke bawah. Program akan langsung meminta input melalui prompt interaktif di bagian output sel.

LAMPIRAN B. Source Code Lengkap

Berikut adalah keseluruhan source code program FSM String Validator yang diimplementasikan pada Google Colab.

```
def cek_fsm(s):
    state = 'S'
    history = ['S']
    alasan = ""
    output_lines = []

    for i, c in enumerate(s):
        if c not in ['0', '1']:
          return "Input tidak valid"

        if state == 'S':
            state = 'A' if c == '0' else 'B'

        elif state == 'A':
            if c == '0':
                state = 'C'
                alasan = "mengandung substring '00'\n"
            else:
                state = 'B'

        elif state == 'B':
            state = 'A' if c == '0' else 'B'

        elif state == 'C':
            state = 'C'

        history.append(state)

    output_lines.append("\nTRACE FSM:")
    output_lines.append(" -> ".join(history))

    if state == 'B':
        output_lines.append("DITERIMA\n")
    else:
        output_lines.append("DITOLAK")
        if alasan:
            output_lines.append(f"Alasan: {alasan}")
        elif s and s[-1] != '1':
            output_lines.append("Alasan: tidak berakhir dengan '1'\n")
```

```
    return "\n".join(output_lines)


# UI
while True:
    s = input("Masukkan string (0 dan 1): ")
    result = cek_fsm(s)

    print(result)

    if "Input tidak valid" in result:
        break

    ulang = input("Ulangi? (y/n): ")

    if ulang.lower() == 'y':
        continue
    elif ulang.lower() == 'n':
        print("Program berhenti")
        break
    else:
        print("inputan gagal")
        break
```